练习 1

- is_prime_helper 是尾递归，因为递归调用仅在 else 分支最后发生，因为 && 运算符求值顺序是从左到右。
- 编写一个与 is_prime_helper_rec 逻辑相同的非尾递归版本 is_prime_rec，找到一个使其栈溢出的参数 x0，并将 x0 作为 is_prime_helper 的参数，观察其是否会栈溢出。如果没有栈溢出，则为尾递归，反之亦然。

```
let rec is_prime_helper_rec x i =
    if i == 1 then
        true
    else
        (is_prime_helper_rec x (i-1)) && (x mod i != 0)
```

练习 2

```
let has_single_elem l =
    match l with
    | [] -> false
    | [_] -> true
    | _ -> false
```

练习 3

```
let rec list_max_iter l m =
    match l with
    | [] -> m
    | x :: xs -> list_max_iter xs (max m x)
```

> list_max_iter 无法还原 list_max 的 failwith

练习 4

```
List.map (fun x -> x * x) your_list
```

练习 5

```
let rec filter f l =
    match l with
    | [] -> []
    | x :: xs ->
        (if (f x) then [x] else []) @ (filter f xs)
```

练习 6

```
let rec count_123 l =
    match l with
    | [] -> 0
    | 1 :: 2 :: 3 :: xs -> 1 + (count_123 xs)
    | x :: xs -> count_123 xs
```

练习 7

```
let rec sum_iter l acc =
    match l with
    | [] -> acc
    | h :: t -> sum_iter t (acc + h)
```

练习 8

```
let rec diff l =
    match l with
    | [] -> []
    | [x] -> []
    | x :: y :: xs -> (y - x) :: (diff (y :: xs))
```

练习 9

```
let rec fold_right f l acc =
    match l with
    | [] -> acc
    | x :: xs -> f x (fold_right f xs acc)
```

练习 10

```
let string_to_char_list str =
    String.fold_right (fun c acc -> c :: acc) str []

let rec rotate_left lst n =
    if n <= 0 then lst
    else
        match lst with
        | [] -> []
        | hd :: tl -> rotate_left (tl @ [hd]) (n - 1)

let rotate_right lst n =
    let len = List.length lst in
    let n = n mod len in
    rotate_left lst (len - n)

let rec smallbf_helper (curr_mem :: rest_mem) (cmd : char list)
offset =
    match cmd with
    | [] -> rotate_right (curr_mem :: rest_mem) offset
    | '+' :: rest -> smallbf_helper (curr_mem + 1 :: rest_mem)
rest offset
    | '>' :: rest -> smallbf_helper (rest_mem @ [curr_mem]) rest
(offset+1)

let smallbf mem cmd_as_str =
    smallbf_helper mem (string_to_char_list cmd_as_str) 0
```